Error bounds on higher order product formulas for translationally-invariant Hamiltonians

March 31, 2026

Abstract

Quantum simulation remains in the frontier of quantum information research due to the ability to simulate systems that classical computers cannot efficiently. Although tremendous efforts have been made to present theoretical errors bounds for product formulae, these theoretical bounds, especially for higher-order formulae, are complicated and unclear. In this paper, we introduce a python package and general classical algorithms that would allow for efficient computation of these theoretical bounds for any Hamiltonian for Pauli Strings and Fermionic Operators. In particular, we take advantage of the symmetry of translationally invariant systems to remove system-size dependence of commutator calculations. In addition to arbitrary Hamiltonians, we design a more sophisticated pipeline to compute higher-order product formula bounds. We consider the application of this package for Pionless-EFTs, where we implement sophisticated graph coloring algorithms for tighter norm calculations. We present time-step and circuit depth calculations for Pionless-EFTs which achieve comparable performance to previous methods. [AMC: this is a good start; let's refine it later]

1 Introduction

Despite their simplicity, product formulae methods for quantum simulation are among the best performing, easiest to implement, and generally most popular algorithms to simulate the evolution of quantum systems. Although their asymptotic performance is worse than other more modern methods, such as quantum signal processing (QSP) [1], empirically we see that their performance for relevant timescales is comparable to or even better than QSP errors. On the other hand, product-formula error bounds often depend on complicated expressions of nested commutators which are difficult to compute, and several of the authors have been personally victimised by these computations. Childs et al. [2] provided an explicit expression for product formulae, but the commutator-norm error scaling dependence on Hamiltonians make the expressions difficult to understand. Watson et al. [3] explicitly wrote out expressions for order $p = 1, 2$ error bounds of pionless-EFTs and did asymptotic analysis for error bounds of nuclear Hamiltonians. Such approaches are unfeasible for higher order Trotter formulas or for more complicated Hamiltonians. Therefore, we introduce generalizable computer algorithms for computation of error bounds. We choose to work with the fermionic operators directly to take advantage the symmetries involved in translation invariance. No coding package to the best of our knowledge is able to compute the error bounds of [2] for translationally invariant Hamiltonians.

In this paper, we provide a general algorithm for computing product-formulae error bounds. These algorithms are applicable to k-local spin Hamiltonians and fermion Hamiltonians. We also provide special support for fermion Hamiltonians consisting of hopping and number operators. We present a framework and python code to handle translation invariance of k-local Hamiltonians for arbitrary lattice size. We implement code which represent Pauli Operators and Fermion Operators symbolically, as well as their respective commutator relations. We provide time-step comparisons between previous methods and present

implementation details compared to [3] for the Pionless-EFT, as well as show plots for hyperparameters that are specific to our implementation.

We organize the rest of our paper as follows. In section 2, we discuss crucial definitions and background for definitions and variables we will reference in the later parts of the paper. In section 3, we write out details of our framework for handling commutator-norm expressions, as well as proofs that our methods are sound. In Section 4, we discuss implementation-specific details of our algorithms and the short-comings of explicitly computing error bounds involving commutator-norms. In Section 5, we show experimental results and plots of our methods. Finally, in Section 6, we summarize our contributions and provide future directions for work.

2 Preliminaries

2.1 Notation and Terminology

We begin with some notational conventions: Throughout, we denote the set of natural numbers $1, 2, \ldots, n$ as $[n]$. Given two positive integers, x, y, we denote their least common multiple (LCM) as $\mathrm{lcm}(x, y)$. For vectors $\boldsymbol{x}$ and $\boldsymbol{y}$ $\mathrm{lcm}(\boldsymbol{x}, \boldsymbol{y})$ denotes the vector of elementwise LCMs. Here, boldface is used to denote a vector. Finally, we consider several different operator norms: $\|\cdot\|$ will denote the operator (spectal) norm, $\|\cdot\|_1$ will denote the 1-norm, $\|\cdot\|_f$ will denote the Frobenius norm, and $\|\cdot\|_\eta$ will denote the *fermionic semi-norm* of an operator for a system with η particles (see Section 3.2 for details) [4].

We will consider Hamiltonians acting on a discrete lattice $\Lambda \subseteq \mathbb{Z}^d$ in d dimensions. Lattice sites will be denoted as vectors $\boldsymbol{x} \in \Lambda$. We enumerate the sites of Λ via a mapping $f : \Lambda \mapsto \{0, 1, \cdots, |\Lambda| - 1\}$, where we refer to the output integers as (site) indices. A generic Hamiltonian of N terms acting on this lattice can be written as

$$H = \sum_{i \in [N]} h_i \tag{1}$$

where each h_i is a Hermitian operator. A Hamiltonian H is k-local if each h_i acts on at most k sites.

In this work, we consider two types of k-local Hamiltonians: Pauli (qubit) Hamiltonians and number-preserving fermionic Hamiltonians. For Pauli Hamiltonians the lattice sites are qubit degrees of freedom and each term takes the form $h_i = c_i \prod_{j \in S_i} h_{ij}$, where $S_i \subset f(\Lambda)$ is the set of site indices on which the operator h_i acts non-trivially and c_i is a real-valued coefficient. Each $h_{ij} \in \{X_j, Y_j, Z_j\}$, where X_j, Y_j, Z_j denote the Pauli operators on acting on site j. For fermionic Hamiltonians, the lattice sites are fermionic modes and the terms h_i are number-preserving fermionic operators. Specifically, we will focus on two-body operators where $h_i = c_i \prod_{j \in S_i \times T_i} u_{ij}$, where $S_i \cup T_i \subset f(\Lambda)$ is the set of site indices that h_i acts non-trivially. We enforce $S_i \cap T_i = \emptyset$, and each $u_{ij} \in \{a^\dagger_{j_1} a_{j_1} I_{j_2}, a^\dagger_{j_1} a_{j_1} a^\dagger_{j_2} a_{j_2}, a^\dagger_{j_1} a_{j_2} + a^\dagger_{j_2} a_{j_1}\}$, where $j := (j_1, j_2)$ and $a^\dagger$ and a are fermionic creation and annihilation operators, respectively.

Our focus is on translation-invariant Hamiltonians. Thus, define the translation operator $\mathcal{T}_{\boldsymbol{v}}(\cdot)$ for some vector $\boldsymbol{v} \in \mathbb{Z}^d$. This operator acts as

$$\mathcal{T}_{\boldsymbol{v}}(h_i) = c_i \prod_{j \in \mathcal{T}_{\boldsymbol{v}}(S_i)} h_{ij}, \tag{2}$$

where $\mathcal{T}_{\boldsymbol{v}}(S_i) := f(f^{-1}(j) + \boldsymbol{v})$ for each index $j \in S_i$; that is, it translates each coordinate vector in Λ associated with $j \in S_i$ by the vector $\boldsymbol{v}$ and then maps the result back to the corresponding index. A Hamiltonian H is translationally invariant if we can express it as

$$H = \sum_{i=1}^{M} \sum_{\boldsymbol{v} \in \boldsymbol{g_i} \circ \mathbb{Z}^d} \mathcal{T}_{\boldsymbol{v}}(h_i), \tag{3}$$

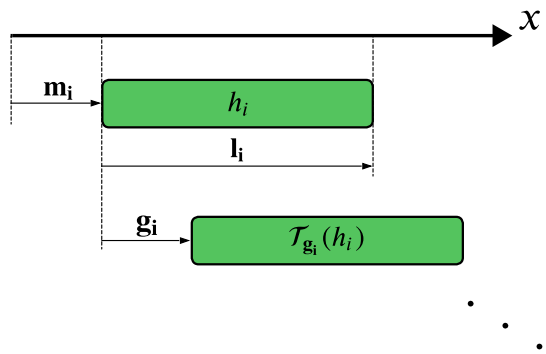

Figure 1: A 1D visualization of a translationally invariant term. Here, h_i is a k-local operator, g_i is the translation vector, m_i denotes the smallest lattice index in which h_i acts on, and l_i is the length of the term i.e. the difference between the largest and smallest lattice indices in which h_i acts non-trivially. Precise definitions of these operators are given in Section 2.1.

where $\circ$ denotes element-wise multiplication, $\boldsymbol{g_i} \in \mathbb{Z}^d$ is the *generator vector* indicating the amount with which the *base term* h_i is translated along the lattice before repeating, and M is the number of base terms in the translationally invariant Hamiltonian. The construction of Eq. (3) is not unique. Given any decomposition Eq. (3), we represent each inner sum of Eq. (3) by the tuple $(h_i, \boldsymbol{g_i})$, where h_i is a base term and $\boldsymbol{g_i}$ is the generator vector that translates the base term across the lattice. For each tuple $(h_i, \boldsymbol{g_i})$, we define $\boldsymbol{m_i}$ as the lattice coordinate of the smallest qubit index $j \in f(\Lambda)$ on which the base term h_i acts non-trivially, i.e. $\min \operatorname{supp}(h_i)$. Then, we can define the *length* $\boldsymbol{l_i}$ of h_i as

$$\boldsymbol{l_i} := \max(\operatorname{supp}(h_i)) - \min(\operatorname{supp}(h_i)) \tag{4}$$

A visualization of these definitions for the 1D case is provided in Fig. 1.

2.2 Hamiltonian Simulation via Product Formulas

Product formulas, or Trotterization, are a standard approach to simulating time evolution of quantum systems. For simplicity, suppose we have a time-independent Hamiltonian H. The key idea of these methods is to approximate the time evolution e^{-itH} via a decomposition into products of smaller, simpler-to-implement unitary evolutions for small time steps.

In particular, suppose we can a decompose the Hamiltonian into Γ terms

$$H = \sum_{\gamma=1}^{\Gamma} H_\gamma, \tag{5}$$

such that the unitaries e^{-itH_γ} can be implemented exactly and efficiently for any desired time t by a simple quantum circuit. Then, the time evolution under H can be approximated via the first order Suzuki-Trotter product formula [5]

$$\mathcal{P}_1(t) := \prod_{\gamma=1}^{\Gamma} e^{-itH_\gamma}. \tag{6}$$

This approximation can be improved by the second-order Suzuki-Trotter formula

$$\mathcal{P}_2(t) := \prod_{\gamma=1}^{\Gamma} e^{-itH_\gamma/2} \prod_{\gamma=\Gamma}^{1} e^{-itH_\gamma/2}, \tag{7}$$

where the reversed indexing on the second product indicates that the order of the product is reversed compared to the first. In turn, one can construct higher-order formulas (for even order, p) via the recursion relation

$$\mathcal{P}_{p+2}(t) := \mathcal{P}_p^2(s_p t)\mathcal{P}_p\big((1-4s_p)t\big)\mathcal{P}_p^2(s_p t), \qquad p \in 2\mathbb{N},\, p \geq 2, \tag{8}$$

where $s_p := (4 - 4^{1/(p+1)})^{-1}$ and $\Upsilon := 2 \times 5^{p/2-1}$ are order-dependent constants.

The error induced by these these product-formula estimates can be upper bounded as [2]:

$$\left\| e^{-itH} - \mathcal{P}_p(t) \right\| \leq 2\Upsilon^{p+1} \frac{t^{p+1}}{p+1} \tilde{\alpha}_{\text{comm}}^{(p)}, \tag{9}$$

where

$$\tilde{\alpha}_{\text{comm}}^{(p)} := \sum_{\gamma_{p+1},\gamma_p,\ldots,\gamma_1=1}^{\Gamma} \left\| [H_{\gamma_{p+1}}[H_{\gamma_p},\ldots[H_{\gamma_2},H_{\gamma_1}]]] \right\|, \tag{10}$$

is a sum of nested commutators of the terms in the Hamiltonian decomposition of Eq. (5).

The bound in Eq. (9) already improves on earlier approaches [CITE] to bounding product formula errors in a variety of ways. However, further tightening of the bounds on the error can be done for the first and second order cases due to their relative simplicity [2]. In particular, for the first order ($p = 1$) Trotter formula:

$$\left\| e^{-itH} - \mathcal{P}_1(t) \right\| \leq \frac{t^2}{2} \sum_{\gamma_1=1}^{\Gamma} \left\| \left[H_{\gamma_1}, \sum_{\gamma_2=\gamma_1+1}^{\Gamma} H_{\gamma_2} \right] \right\|, \tag{11}$$

and, for the second order ($p = 2$) formula:

$$\begin{aligned} &\left\| e^{-itH} - \mathcal{P}_2(t) \right\| \\ &\quad \leq \frac{t^3}{12} \sum_{\gamma_1=1}^{\Gamma} \left\| \left[\sum_{\gamma_3=\gamma_1+1}^{\Gamma} H_{\gamma_3}, \left[\sum_{\gamma_2=\gamma_1+1}^{\Gamma} H_{\gamma_2}, H_{\gamma_1} \right] \right] \right\| + \frac{t^3}{24} \sum_{\gamma_1=1}^{\Gamma} \left\| \left[H_{\gamma_1}, \left[H_{\gamma_1}, \sum_{\gamma_2=\gamma_1+1}^{\Gamma} H_{\gamma_2} \right] \right] \right\|. \end{aligned} \tag{12}$$

[COMMENT ON THE FACT THESE BOUNDS ARE LOOSE IN PRACTICE + REFERENCES]

Using error bounds of the sort described above, it is relatively straightforward to upper bound the number of time steps r required to simulate e^{-itH} up to a fixed error ϵ_{prod}. When comparing different product formulae r can serve as a useful proxy for circuit depth. In particular, consider dividing the time evolution into r steps and, then, apply the triangle inequality:

$$\epsilon_{\text{prod}} := \left\| e^{-itH} - \mathcal{P}_p(t) \right\| \leq r \left\| e^{-i(t/r)H} - \mathcal{P}_p(t/r) \right\|. \tag{13}$$

Noting that all three of the error bounds in Eqs. (9), (11) and (12) take the form $\left\| e^{-itH} - \mathcal{P}_p(t) \right\| \leq t^{p+1} F(p)$ for some function $F(p)$, we can substitute any one of those bounds into the right hand side of Eq. (13). Thus, we find the required number of time steps to achieve the desired error is bounded by

$$r \leq \left[\frac{t^{p+1}}{\epsilon_{\text{prod}}} F(p) \right]^{\frac{1}{p}}. \tag{14}$$

The $\sim t^{1+1/p}$ dependence implies that for some sufficiently large evolution time t higher order product formulas always yield equivalent error for fewer steps r, but the crossover point depends heavily on the function $F(p)$. Enabling rapid computation of these functions $F(p)$ for translation-invariant Hamiltonians—in order to find these crossover points—is the key goal of this work.

2.3 The Fermionic Semi-norm

Under the assumption that we know the number of fermions in the state, the operator norms in the Trotter error bounds can be replaced with the so-called fermionic semi-norm for number-preserving fermionic Hamiltonians. Unlike the more general bounds of the previous section, these bounds are state-dependent. Leveraging, this prior knowledge allows for tighter bounds.

Definition 1 (Fermionic Semi-Norm (Section 2.3 of Ref. [4])). *Let X be a number-preserving fermionic operator and let $|\psi_\eta\rangle$ and $|\phi_\eta\rangle$ be normalized states with exactly η fermions. Then, the fermionic semi-norm of X is*

$$\|X\|_\eta = \max_{|\psi_\eta\rangle, |\phi_\eta\rangle} |\langle\psi_\eta| X |\phi_\eta\rangle|. \tag{15}$$

Furthermore, if X is Hermitian, then

$$\|X\|_\eta = \max_{|\psi_\eta\rangle} |\langle\psi_\eta| X |\psi_\eta\rangle|. \tag{16}$$

In this setting, the crucial task is therefore to bound the fermionic semi-norm of nested commutators of number-preserving fermionic operators. Two approaches to bounding this quantity were provided in Ref. [4]. Here, we will use a generic upper bound from Ref. [3] that leverages a decomposition of a fermionic operator into a sum of terms acting on non-intersecting subsets of lattice sites:

Theorem 1 (Theorem 34 of Ref. [3]). *Consider a set of fermionic modes, $f(\Lambda) = \{0, 1, \cdots, |\Lambda| - 1\}$. Let $\boldsymbol{i} = (i_1, i_2, \ldots, i_{k_{\boldsymbol{i}}})$ denote a tuple of $k_{\boldsymbol{i}}$ indices for some constant $k_{\boldsymbol{i}}$, and let $\Omega = \{\boldsymbol{i}_1, \boldsymbol{i}_2 \ldots\}$ be a set of such tuples such that no tuple shares indices with any other tuple—thus, $\forall\, \boldsymbol{i}_a, \boldsymbol{i}_b \in \Omega$ with $a \neq b$, $\boldsymbol{i}_a \cap \boldsymbol{i}_b = \emptyset$.*

Given a number-preserving fermionic operator of the form

$$X = \sum_{\boldsymbol{i}\in\Omega} J_{\boldsymbol{i}} h_{\boldsymbol{i}}, \tag{17}$$

where each $h_{\boldsymbol{i}}$ is a number-preserving fermionic operator acting on the fermionic modes in $\boldsymbol{i} \subset f(\Gamma)$, the fermionic semi-norm of X can be bounded as

$$\|X\|_\eta \leq J_{\max} \min\left\{ \left\lceil \frac{\eta}{\lceil k_{\min}/2 \rceil} \right\rceil, |\Omega| \right\}, \tag{18}$$

where $k_{\min}$ is the minimum locality of $h_{\boldsymbol{i}}$ and $J_{\max} = \max_{\boldsymbol{i}\in\Omega}\{|J_{\boldsymbol{i}}|\}$.

In the context of Trotter error bounds the operator X will be a nested commutator of number-preserving fermionic Hamiltonians. To apply the bound, X must be split into terms acting on non-intersecting tuples of sites. One of the key elements of our work is an automated way to perform this decomposition for translation-invariant number-preserving fermionic operators.

2.4 Trotter Error Bounds for Translation Invariant Hamiltonians

Whether we are considering qubit or fermionic degrees of freedom, the bounds developed in the previous two sections do not leverage translation invariance. Indeed, the decomposition of a Hamiltonian H for the purposes of Trotterization given in Eq. (5) is distinct from the expression for a translation invariant Hamiltonian in Eq. (3). Thus, to find Trotter error bounds for a translationally invariant Hamiltonian it is crucial that when we partition the Hamiltonian into efficiently implementable subsets $\{H_1, H_2, ...H_\Gamma\}$ we preserve the translation invariance of the full Hamiltonian. We formalize this requirement as follows:

Definition 2. *Consider a decomposition*

$$H = \sum_{\gamma=1}^{\Gamma} H_\gamma = \sum_{\gamma=1}^{\Gamma} \sum_{i \in M_\gamma} \sum_{\boldsymbol{v} \in \boldsymbol{g_i} \circ \mathbb{Z}^d} \mathcal{T}_{\boldsymbol{v}}(h_i), \tag{19}$$

of a translation-invariant Hamiltonian, where $\bigcup_{\gamma=1}^{\Gamma} M_\gamma$ *is a partition of the number of base terms* $M = \sum_{\gamma=1}^{\Gamma} |M_\gamma|$, h_i *for* $i = 1, 2, ..., |M_\gamma|$ *are the base terms, and* $\boldsymbol{g_i} \in \mathbb{Z}^d$ *for* $i = 1, 2, ..., |M_\gamma|$ *are the generator vectors. A decomposition is* ***valid*** *if, for all* $\gamma \in \Gamma$

$$[\mathcal{T}_{\boldsymbol{v}}(h_i), \mathcal{T}_{\boldsymbol{w}}(h_j)] = 0$$

for all base terms $h_i, h_j \in M_\gamma$ *and all corresponding translation vectors* $\boldsymbol{v} \in \boldsymbol{g_i} \circ \mathbb{Z}^d, \boldsymbol{w} \in \boldsymbol{g_j} \circ \mathbb{Z}^d$.

This definition ensures that base terms grouped into efficiently implementable sums of terms $H_\gamma = \sum_{i \in M_\gamma} \sum_{\boldsymbol{v} \in \boldsymbol{g_i} \circ \mathbb{Z}^d} \mathcal{T}_{\boldsymbol{v}}(h_i)$ by virtue of these sub-Hamiltonians consisting only of mutually commuting sets of translation invariant terms. Ideally, we select a valid decomposition where Γ and M are minimized as this will lead to the tighest possible bounds. However, any valid decomposition yields valid bounds and, in practice, finding a global minimum can be difficult for complicated Hamiltonians.

3 Computing Error Bounds for Translationally Invariant Hamiltonians

As described in Section 2, computing product formula error bounds reduces to finding the norm of a nested commutator of efficiently implementable Hamiltonian terms with the "level" of the nesting depending on the order p of the product formula.

Given a valid decomposition, we can ensure these commutators are themselves translationally invariant. To this end, we will first demonstrate how to compute a single commutator between two translationally invariant terms, which we then use as a subroutine to compute nested commutators. Under the assumption that the Hamiltonians are k-local and translationally invariant, we can reduce the computation of an infinite number of commutators that arise on an infinite lattice to a finite number of commutators which are, themselves, translationally invariant. While our bounds apply to (sufficiently large) finite lattices, in what follows we assume an infinite lattice for ease of presentation.

[Finish summary once this section is fully cleaned up]

3.1 Commutators of Translationally Invariant Operators

Let $A = (a, \boldsymbol{g_a})$ and $B = (b, \boldsymbol{g_b})$ be translation-invariant operators. Here, a, b are the associated base terms and $\boldsymbol{g_a}, \boldsymbol{g_b} \in \mathbb{Z}^d$ are generator vectors. That is, A, B can be written as

$$A = \sum_{\boldsymbol{v} \in \boldsymbol{g_a} \cdot \mathbb{Z}^d} \mathcal{T}_{\boldsymbol{v}}(a) \tag{20}$$

$$B = \sum_{\boldsymbol{v} \in \boldsymbol{g_b} \cdot \mathbb{Z}^d} \mathcal{T}_{\boldsymbol{v}}(b). \tag{21}$$

The commutator $C := [A, B]$ is, itself, a linear combination of translation invariant terms. In particular, explicitly working out the commutator of the sums in Eqs. (20) and (21), one can write C as a sum of M_c translation invariant terms $C_k = (c_k, \boldsymbol{g_c})$ for $k \in \{1, 2, \cdots M_c\}$,

$$C = \sum_{k=1}^{M_c} \sum_{\boldsymbol{v} \in \boldsymbol{g_c} \circ \mathbb{Z}^d} \mathcal{T}_{\boldsymbol{v}}(c_k), \tag{22}$$

where $\boldsymbol{g}_c = \mathrm{lcm}(\boldsymbol{g_a}, \boldsymbol{g_b}) \in \mathbb{Z}^d$ is a (k-independent) generator vector, and the base terms are given by

$$c_k = [\mathcal{T}_{\boldsymbol{v}_a}(a), \mathcal{T}_{\boldsymbol{v}_b}(b)], \tag{23}$$

where $(\boldsymbol{v_a}, \boldsymbol{v_b})$ are from the set

$$S = \{(\boldsymbol{v_a}, \boldsymbol{v_b}) \in (\boldsymbol{g_a} \circ \mathbb{Z}^d, \boldsymbol{g_b} \circ \mathbb{Z}^d) \mid \min(\boldsymbol{v_a}, \boldsymbol{v_b}) < \boldsymbol{g}_c\}. \tag{24}$$

Here, the constraint $\min(\boldsymbol{v_a}, \boldsymbol{v_b}) \leq \boldsymbol{g}_c$ is to be interpreted component-wise. Note $M_c = |S|$. A 1D example is shown in Fig. 2.

It is straightforward to automate the computation of the set S, and thus the set of M_c translation invariant terms that make up the commutator C. As the result of this computation is a set of translation invariant terms, in the context of Trotter error bounds of a Hamiltonian written in a valid decomposition (Definition 2), one can recursively apply this procedure in order to decompose nested commutators of translation invariant terms into their own sums of translation invariant terms.

In [REF] we provide code to perform these commutator calculations and to bound the norms of the resulting operators to calculate Trotter error bounds for translation-invariant Hamiltonians. In the case of Pauli Hamiltonians this procedure is straightforward. For number-preserving fermionic Hamiltonians, further algorithmic development is necessary to automate the application of bounds on the fermionic seminorm along the lines of Theorem 1. In the next section, we turn to this problem.

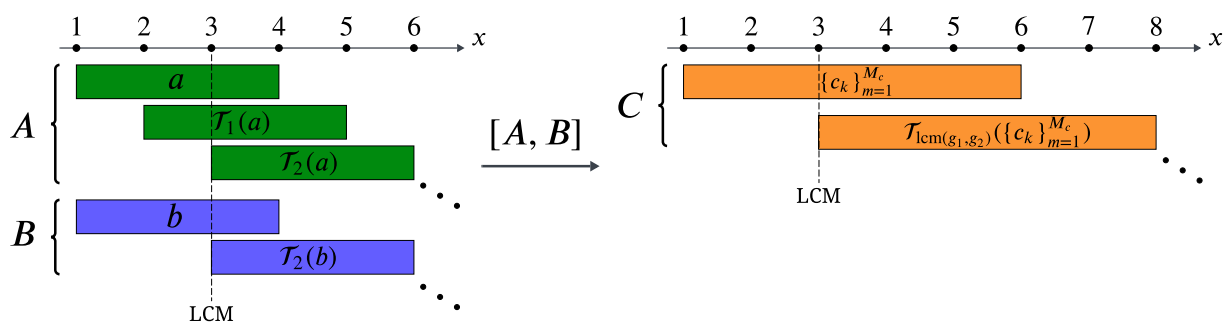

Figure 2: A 1-dimensional visualization of a commutator of two translationally invariant operators inducing another translationally invariant operator. In this example, we consider the commutator calculation of operator $(h_1, g_1 = 1)$, labeled green, with operator $(h_2, g_2 = 2)$, labeled maroon. Here, $\{c_k\}_{m=1}^{M_c} = [h_1, h_2] + [h_1, \mathcal{T}_2(h_2)] + [\mathcal{T}_1(h_1), h_2] + [\mathcal{T}_1(h_1), \mathcal{T}_2(h_2)] + [\mathcal{T}_2(h_1), h_2]$. We highlight that for these computations, we only need to compute commutators where at least one of the operators have an offset less than the LCM. All byproducts of the commutator calculations have translation of the LCM of the h_1 and h_2 terms i.e. $\mathrm{lcm}(g_1, g_2) = 2$.

.

3.2 Upper Bounds via the Fermionic Semi-Norm

Theorem 1 is the basis for upper bounding the fermionic seminorm of the translation invariant operators that emerge from the nested commutator calculations described in the previous section in the context of number-preserving fermionic Hamiltonians. In particular, an immediate consequence of Theorem 1 is that given a set of translation invariant number-preserving fermionic operators T (in our case, obtained via a nested commutator calculation), to bound the fermionic seminorm of

$$H_T = \sum_{(h,\boldsymbol{g}) \in T} \sum_{\boldsymbol{v} \in \boldsymbol{g} \circ \mathbb{Z}^d} \mathcal{T}_{\boldsymbol{v}}(h) = \sum_j h_j \tag{25}$$

at fixed particle number η it is sufficient to divide the terms in H_T into subsets acting on disjoint sets of fermionic sites. In Eq. (25) the operators h_j are implicitly defined as the set of all terms that arise from translations of the base terms h. We do not put an upper bound on this sum because, as discussed above, we develop our results for an infinite lattice. Once we make use of translation invariance we will only need to consider a finite number of terms and the resulting bounds will apply for all sufficiently large finite lattices.

To accomplish the task of dividing the terms $\{h_j\}$ into subsets acting on disjoint sites, consider an interaction graph $G = (V_G, E_G)$ with a vertex set

$$V_G = \{h_j\} \tag{26}$$

(i.e., the vertices correspond to the individual fermionic operators in H_T) and an edge set

$$E_G = \{(h_i, h_j) \mid \mathrm{supp}(h_i) \cap \mathrm{supp}(h_j) \neq \emptyset\} \tag{27}$$

(i.e., there exists an edge between two vertices if and only if the corresponding operators act non-trivially on the same lattice sites in Λ). The fermionic seminorm of H_T for fixed particle number η can be then be upper-bounded in terms of the chromatic number of G [CITE], as described in the following lemma.

Lemma 1. *Given a number-preserving fermionic Hamiltonian $H_T = \sum_i h_i = \sum_i c_i \prod_{j\in\mathrm{supp}(h_i)} h_{ij}$, where $\mathrm{supp}(h_i)$ is the set of site indices on which the operator h_i acts non-trivially and c_i are real-valued coefficients, and an associated interaction graph G, the fermionic semi-norm of H_T for fixed particle number η is bounded as*

$$\|H_T\|_\eta \leq \chi(G) \left\lceil \frac{2\eta}{k_{\min}} \right\rceil \max\{c_i\}$$

where $\chi(G)$ is the chromatic number of G and $k_{\min} = \min\{|\mathrm{supp}(h_i)|\}$.

Proof. By construction, a coloring of G divides the associated vertices into disjoint sets of fermionic terms. Let $S = \{S_j\}$ where $S_j \subset V_G$ are the disjoint sets of vertices. If the coloring is minimal then $|S| = \chi(G)$. We can apply the triangle inequality on $\|H_T\|_\eta$ with respect to the sets S_j and then use Theorem 1 to bound the fermionic seminorm of the sets. In particular,

$$\|H_T\|_\eta = \sum_{i\in[|S|]} \left\| \sum_{j\in S_i} h_j \right\|_\eta \leq \sum_{i\in[|S|]} \max\{c_j\}_{j\in S_i} \left\lceil \frac{2\eta}{k_{\min}} \right\rceil = \chi(G) \left\lceil \frac{2\eta}{k_{\min}} \right\rceil \max\{c_j\}.$$

□

This formulation gives an upper bound as a function of the chromatic number of graph G. However, G has infinitely many vertices, making a direct coloring of G not well defined. To remedy this, we show how the an infinitely large graph G of a translationally invariant H induces a finitely large graph $\widetilde{G}$ i.e. the coloring of $\widetilde{G}$ describes a coloring for G. Crucially, due to translation invariance, finding a coloring of the individual G does not require constructing or iterating over the full graph (which has an arbitrarily large number of vertices). We explicitly define $\widetilde{G}$ as follows:

Definition 3. *Let $S \coloneqq \{(h_i, \boldsymbol{g_i})\}_{i=1}^n$ be a collection of translationally invariant terms. Define $\mathrm{lcm}(S) \coloneqq \mathrm{lcm}\{\boldsymbol{g_i}\}_{i=1}^n$ as the component-wise lowest common multiple (LCM) of the generator vectors $\boldsymbol{g_i}$. Let $\boldsymbol{\delta} \in (\mathbb{Z}^+)^d$ be defined component-wise as*

$$\boldsymbol{\delta} = \left\lceil \frac{\max\{\boldsymbol{l_i}\}_{i=1}^n}{\mathrm{lcm}(S)} \right\rceil,$$

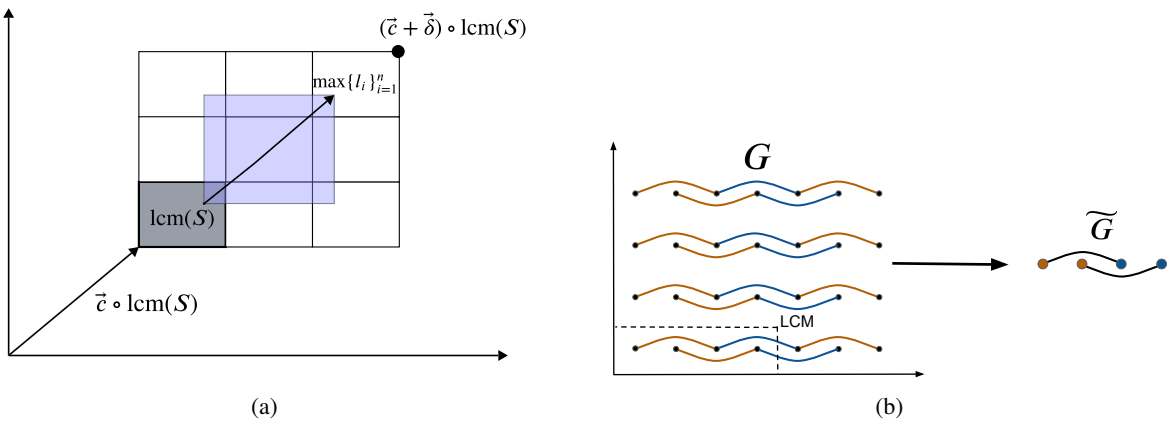

Figure 3: Figure (a) demonstrates the set of vertices used in the construction of $\widetilde{G}$ in Definition 3. Here, $\boldsymbol{c}\circ \mathrm{lcm}(s)$ represents the smallest multiple of the lcm box such that a translation of each $h_i \in S\ \forall i$ exists inside the component-wise interval $[\boldsymbol{c}\circ \mathrm{lcm}(S), (\boldsymbol{c}+\mathbf{1})\circ \mathrm{lcm}(S))$, visualized by the light-gray box in figure (a). To avoid a graph construction for $\widetilde{G}$ that contains nodes with self-edges, we must construct the size of $\widetilde{G}$ larger than the h_i with the largest l_i. Mathematically, this is $(\boldsymbol{c}+\boldsymbol{\delta})\circ \mathrm{lcm}(S)$. Combining these two ideas, Figure (b) is an example of the construction of $\widetilde{G}$ for Hopping-to-Hopping commutators for the 2D Pionless-EFT. We emphasize that a graph G induces $\widetilde{G}$.

where $\boldsymbol{l_i}$ is the length of h_i, and let $\boldsymbol{c}\in(\mathbb{Z}^+)^d$ be defined as

$$\boldsymbol{c} = \left\lceil \frac{\max\{\boldsymbol{m_i}\}_{i=1}^n}{\mathrm{lcm}(S)} \right\rceil,$$

where $\boldsymbol{m_i}$ is the lattice coordinate of the smallest qubit index $j\in f(\Lambda)$ on which the term h_i acts non-trivially. Then, the vertices of $\widetilde{G}$ are given by

$$V_{\widetilde{G}} = \{\mathcal{T}_{(\boldsymbol{c}+\boldsymbol{z})\circ \mathrm{lcm}(S)}(h_j) \mid \min(\mathrm{supp}(h_j)) \in [\boldsymbol{c}\circ \mathrm{lcm}(S), (\boldsymbol{c}+\mathbf{1})\circ \mathrm{lcm}(S)),\ z_i\in[0,\delta_j)\}$$

and the edges of $\widetilde{G}$ are given by

$$E_{\widetilde{G}} = \{(u,v) \mid \mathrm{supp}(u)\ \cap \mathrm{supp}(v) \neq \emptyset\}.$$

The intuition for this definition is as follows: first, subdivide the lattice $\Lambda\subseteq\mathbb{Z}^d$ into rectangular subsets of lattice points whose dimensions are given by $\mathrm{lcm}(S)$. We call this collection of lattice sites an *lcm box* and label each box by its starting location on the lattice (i.e. the lattice coordinate nearest the origin). Then, $\boldsymbol{c}\circ \mathrm{lcm}(s)$ gives the starting location of the smallest multiple of the lcm box such that a translation of each base term $h_i\in S\ \forall i$ exists inside the component-wise interval $[\boldsymbol{c}\circ \mathrm{lcm}(S), (\boldsymbol{c}+\mathbf{1})\circ \mathrm{lcm}(S)]$, as depicted by the light-gray box in Fig. 3(a). All terms that start in this lcm box are included in $V_{\widetilde{G}}$. Then, to avoid a graph construction for $\widetilde{G}$ that contains vertices with self-edges, we also add all terms to $\widetilde{G}$ that start in lcm boxes up to that labeled by $(\boldsymbol{c}+\boldsymbol{\delta}-1)\circ \mathrm{lcm}(S)$, as shown in Fig. 3(a). The vector $\boldsymbol{\delta}$ is chosen based off the the base term h_i with the largest length l_i. A simple example is depicted in Fig. 3.

The construction of $\widetilde{G}$ from any translationally invariant graph G allows us to bound the quantity $\chi(G)$ in Lemma 1. Lemma 1 uses Theorem 1 to provide an upper bound on the fermionic semi-norm as a function of the chromatic number of the graph G. A minimal coloring is not needed as the chromatic number in Lemma 1 can, in turn, be upper bounded by the cardinality of any color set S of G. To find such color sets we consider two approaches: a computationally cheap method that considers applying Lemma 1 individually to each

set of translationally invariant terms, and a more sophisticated, but also more computationally expensive, coloring strategy. To interpolate with respect to the associated efficiency-precision tradeoff between these two methods we will introduce a hyperparameter—the threshold, τ—that selects between the two methods.

We first consider the simple and efficient brute-force coloring algorithm that colors each translationally invariant term with separate sets of colors. This method can be interpreted as taking the triangle-inequality of H_T into individual translationally-invariant terms before applying Lemma 1:

$$\|H_T\|_\eta \leq \sum_{(h,\boldsymbol{g})\in T} \left\| \sum_{\boldsymbol{v}\in \boldsymbol{g}\circ\mathbb{Z}^d} \mathcal{T}_{\boldsymbol{v}}(h) \right\|_\eta \tag{28}$$

Let $G_{(h,\boldsymbol{g})} \subset G_T$ be the interaction graph of the terms in $\sum_{\boldsymbol{v}\in \boldsymbol{g}\circ\mathbb{Z}^d} \mathcal{T}_{\boldsymbol{v}}(h)$. It is apparent that the norms $\left\|\sum_{\boldsymbol{v}\in \boldsymbol{g}\circ\mathbb{Z}^d} \mathcal{T}_{\boldsymbol{v}}(h)\right\|_\eta$ can be computed by applying Lemma 1 to these terms and their associated graphs $G_{(h,\boldsymbol{g})}$ and summing the results.

I'm here————-

Algorithm 1 (Brute-Force Coloring)**.**

Inputs: The set $T = \{(h_i, \boldsymbol{g_i})\}$ Outputs: An upper bound approximation of $\chi(G)$

1. *Initialize total sum $S = 0$.*
2. *For all translationally invariant operators in H,*
   a. *Component-wise, compute the vector $\boldsymbol{C} = \lfloor(\boldsymbol{l_i} - 1)/\boldsymbol{g_i}\rfloor + 1$*
   b. *$S = S + \prod \boldsymbol{C}$, where $\prod \boldsymbol{C}$ is the product of all components of $\boldsymbol{C}$.*
3. *Return S*

The coloring of each translationally invariant term separately avoids pairwise comparisons that would require quadratic time complexity. In addition, the brute-force coloring algorithm avoids the explicit construction of a graph. However, this upper bound is conservative. For example, suppose $t_1 := \sum_{v\in \boldsymbol{z_1}\circ\mathbb{Z}^d} \mathcal{T}_v(h_1)$ and $t_2 := \sum_{v\in \boldsymbol{z_2}\circ\mathbb{Z}^d} \mathcal{T}_v(h_2)$ are two translationally invariant terms and $\mathrm{supp}(t_1) \cap \mathrm{supp}(t_2) = \emptyset$. Then, the direct application of Lemma 1 would be strictly better than Algorithm 1. To ameliorate the looseness of the brute-force coloring, we introduce a more sophisticated coloring which constructs G and colors it directly.

By construction, we can rewrite any $S = \{(h_i, \boldsymbol{g_i})\}_{i=1}^n$ as $S = \{(h_j, \mathrm{lcm}(S))\}_{j=1}^{|\widetilde{G}|}$ where $\{h_j\}_{j=1}^{|\widetilde{G}|}$ represents the set of all operators in $\widetilde{G}$. We denote each $h \in \widetilde{G}$ as a **representative** of G. We define $\tilde{h}$ as the representative of any $h \in G$. To align the construction of $\widetilde{G}$ with the separation of G into sets where each operator acts on a different of qubits, we seek to draw edges between any two operators that act on a similar qubit. It is crucial we also consider when representative operators and non-representative operators act on the same qubit. We describe an algorithm which addresses all of these as follows:

Algorithm 2 (Finding Independent Sets of Translation Invariant Fermionic Operators)**.** *Input: A set of tuple representations $\{(h_i, \boldsymbol{g_i})\}_{i=1}^n$*
Output: An upper bound approximation of $\chi(G)$.

1. *Construct a graph $\widetilde{G}$.*
2. *Add edges to $\widetilde{G}$ according to the following rules: For $u, v \in G$*

- *If* $u, v \in \widetilde{G}$ *and* $\mathrm{supp}(u) \cap \mathrm{supp}(v) \neq \emptyset$ *add* (u, v) *to* $E(\widetilde{G})$
- *If* $u \in \widetilde{G}$ *and* $v \notin \widetilde{G}$ *and* $\mathrm{supp}(u) \cap \mathrm{supp}(v) \neq \emptyset$ *add* $(u, \tilde{v})$ *to* $E(\widetilde{G})$

3. *Vertex color the graph* $\widetilde{G}$.

4. *Assign all* $u \in G$ *to the set associated with the color of its representative* $\tilde{u} \in \widetilde{G}$. *These sets will consist of disjoint operators.*

5. *For each set of disjoint operators, use theorem 1 to compute norms. Sum all these norms together and return.*

A visualization for 2 is illustrated in Fig. 3 for $D = 2$. The key step in the algorithm is Step 2. By construction, these rules ensure that if $(u, v) \in E(G)$ (i.e. if two terms $u, v \in T[O]$ intersect) then their representative will share an edge in $E(\widetilde{G})$. The construction of the additional edges in step 2 of the algorithm ensure that no self-edges between any node can occur. We now put everything together to describe an explicit algorithm for computing α_{comm} (Reference Eq. (10)).

Algorithm 3 (General algorithm for numerically computing α_{comm})**.**
Inputs: Hamiltonian H, order of approximation p, size threshold τ

1. *Construct a valid decomposition* $\{H_i\}_{i=1}^{\Gamma}$ *of* H *(Reference Definition 2).*

2. *Use Eq. (22) to calculate the nested commutator component of* α_{comm}*, given order of approximation* p*. Define*

$$G_{(\gamma_1-1)\cdot(\gamma_2-1)\cdot\ldots\cdot(\gamma_p-1)\cdot(\gamma_{p+1}-1)} \coloneqq [H_{\gamma+1}, [H_{\gamma}, \ldots [H_{\gamma_2}, H_{\gamma_1}]]] \; \forall \gamma_{p+1}, \gamma_p, \ldots, \gamma_2, \gamma_1$$

   Save these calculations separately.

3. *For each* G_i*, compute approximation* $\widehat{\chi(G_i)}$ *using Algorithm 1 if* $|G_i| \leq \tau$*, otherwise use Algorithm 2. Use* $\widehat{\chi(G_i)}$ *as* $\chi(G_i)$ *in Lemma 1 to compute* $\|G_i\|_{\eta}$.

4. *Return* $a_{comm} \coloneqq \sum_i G_i$

4 Implementation Details

In this section, we discuss design choices related to our implementation of the algorithms above [CITE CODE].

4.1 Design Choices

In this package, we utilize the programming language python. For the operators in our package, We implement all the operators symbolically to avoid the exponential scaling of the matrix size of operators. Each operator is represented by a hashmap between lattice locations and the symbolic operator. We have provided support for Pauli strings and the Pionless-EFT. To make any operator translationally invariant, we define a translation invariant class, and store the operator in a list. We utilize lists in our python package to represent sums of operators. In this way, we can represent any arbitrary sum of translationally invariant operators in a single python class. To avoid duplicate calculations of commutators, for the general bound Eq. (9), entire orders of commutators are computed one at a time. As a result, all nested commutators are computed before any norm calculations are being done. This approach sacrifices memory efficiency for computational efficiency. To

compute high-order p computations, all commutator computations for all orders smaller than p are required. Thus, we include support to save commutator computations for an order smaller than p in addition to the original pth order computation. The tighter formulations of $p = 1$ and $p = 2$ commutators are hard-coded, and can be specified as a hyperparameter input into the package. The independence of commutator-norm calculations without each other allows us to parallelize computations. In practice, only a small number of the commutators are large, so parallelization of commutators would provided limited benefit. Therefore, we forgo parallelization for commutator calculations and only parallelize norm calculations. For the coloring of our graph $\widetilde{G}$ in Algorithm 2, we use the greedy coloring algorithm because of its efficiency compare to other coloring algorithms. To implement this, we utilize networkx's [6] greedy_color() function. In our greedy coloring, we traverse the nodes with the highest-degree's first. We choose to use this highest-degree strategy to keep our algorithm deterministic. In addition, for more dense graphs, the highest-degree strategy performs well empirically [AZ: Reference needed]. The constants in front of each Product Formula Bounds are functions of the order p. Because there is no dependence on p in any of the norm calculation algorithms or the circuit depth calculations, we compute the constants right after commutator calculations. We refer further details of our implementation to our github repository.

5 Example: Pionless Nuclear Effective Field Theory

As an application of our results we consider error bounds for Hamiltonian simulation of a simple nuclear physics model as considered in Ref.s In nuclear physics, the Pionless-EFT Hamiltonian is a theoretical framework describing the interactions between nucleons. We consider a discrete lattice in 3 dimensions. We consider the definition of the Pionless-EFT Hamiltonian given by Roggero [7] and Watson [3]

$$H = -h \sum_{\langle x,y \rangle} \sum_{\sigma} [a_x^\dagger(\sigma) a_y(\sigma) + a_y^\dagger(\sigma) a_x(\sigma)] + 6h \sum_x \sum_\sigma N_x(\sigma) \tag{29}$$

$$+ \frac{C_0}{2} \sum_x \sum_{\sigma \neq \sigma'} N_x(\sigma) N_x(\sigma') \tag{30}$$

$$+ \frac{D_0}{6} \sum_x \sum_{\sigma \neq \sigma' \neq \sigma''} N_x(\sigma) N_x(\sigma') N_x(\sigma'') \tag{31}$$

where σ is the spin, and $\langle x, y \rangle$ represents nearest neighbor points and h, C_0 and D_0 are low energy constants.

The simplicity of the Pionless-EFT makes it an ideal candidate for the application of our algorithm. We describe the efficient decomposition of the Pionless-EFT in Appendix A.1, composed of 38 base terms total. We use this description as input into our general algorithm.

5.1 Limitations

One can see that the number of commutator computations in the general product formula error bound Eq. (9) scales exponentially in p. This poor scaling leads to a cascading effect towards the norm calculations that further hinder the algorithm's potential. Here, we discuss these limitations.

5.1.1 Time Complexity Issues

The exponential scaling of the algorithm causes higher-order Product Formula Bounds calculations to be intractible. The algorithm for $p = 2$ took around 20 seconds, while for $p = 4$ commutators of the Pionless-EFT, Step 2 of Algorithm 3 took around 3 hours. [AZ: Should we extrapolate this too?] This suggests that $p = 6$ commutators are out of reach from a time-complexity standpoint.

5.1.2 Memory Limitations

In our implementation, we choose to compute entire orders of commutators at the same time. Consequently, this results in poor memory complexity due to the requirement to store each commutator calculation. Even so, it is reasonable that for higher-order calculations, excessive amounts of memory are required. For $p = 4$ commutators, empirical analysis suggests that 500 gigabytes of RAM are required to store all the resulting base terms. This analysis suggests that $p = 6$ commutator calculations are not feasible at all. Moreover, in the graph coloring algorithm, the creation of each graph described in Algorithm 2 has quadratic memory scaling. The additional overhead of constructing an explicit graph for each commutator output compounds the memory issues of this algorithm. Therefore, the introduction of the brute-force coloring algorithm was necessary to save memory usage.

5.2 Results

Here, we present the application of our algorithms to the Pionless-EFT. Due to the intractability of $p = 6$ order commutators, we compute $p = 1, 2, 4$ error bounds for the general algorithm Eq. (9). In addition, we present computations for the tighter $p = 1, 2$ Product Formulae Error Bounds in Eq. (11), Eq. (12). Here, we present comparisons to previous estimates of α_{comm}. We provide plots to show the effects of changing the threshold value M in Algorithm 3 for circuit depths in the case of the Pionless-EFT Hamiltonian. We compare these computations to error bounds derived from α_{comm} values in Watson et. al [3].

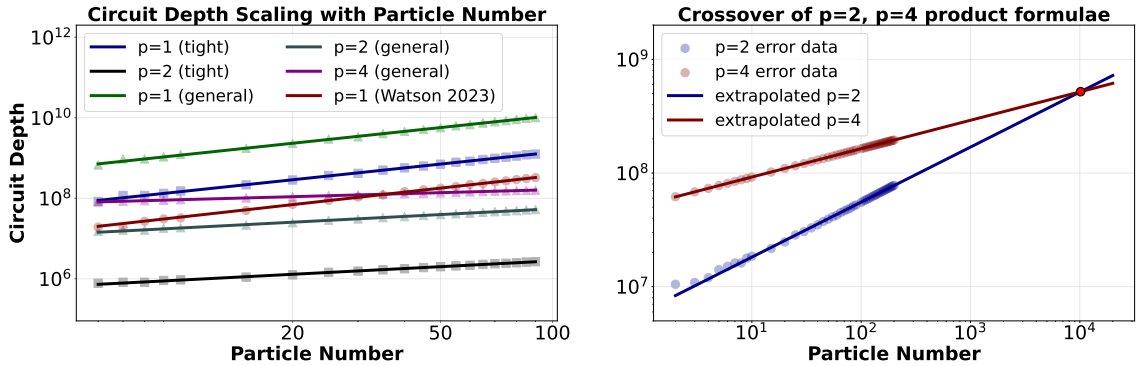

Figure 4: (Left) Error bound computations of order $p = 1, 2, 4$ commutators for the general bound (Eq. (9)), order $p = 1$ (Eq. (11)) and $p = 2$ (Eq. (12)) tighter product formulae bounds, and $p = 1$ error bounds interpolated from $p = 1$ α_{comm} value in Watson et. al [3] on systems with $\eta = 2, 10, 30, 50, 70, 90$ fermions. (Right) A plot showcasing the crossover between lines interpolated from $p = 2$ and $p = 4$ error bound data, computed using the general bound (Eq. (9)). The crossover occurs at $\eta \approx 1.0 \times 10^4$ particles.

Compared to Watson et al. [3], our computer-calculated bounds produce worse $p = 1$ error bounds. This discrepancy can be explained by our choice to produce algorithms that are generalizable to any Hamiltonian rather than solely considering the Pionless-EFT. The computed $p = 4$ error bounds incurs significant overhead. While the crossover between $p = 4$ and $p = 1$ error bounds occurs at around $\eta = 150$ particles, the crossover from $p = 4$ over $p = 2$ product-formulae error bounds (as determined via extrapolation of power law fits to the computed data) occurs at $\eta \approx 1.0 \times 10^4$. While the analytic bounds being compared are not tight, this strongly suggests that for realistic numbers of nucleons for many applications, there is little need to move beyond $p = 2$ product formulae. Furthermore, the 3D pionless EFT considered has only 38 base terms. For more complicated Hamiltonians, such as the pionful EFT, we expect the crossover point between $p = 4$ and $p = 2$ to increase.

In Section 3.2, we introduce the idea of a threshold for graph-coloring computations. We present the effect of the threshold for $p = 1, p = 2$, and $p = 4$ general error bounds in Fig. 5. For these plots, we arbitrarily fix $\eta = 100$ particles. The most improvement can be seen for smaller threshold values. This suggests that the sophisticated graph coloring leads to major improvement for the smaller graphs produced from hopping-to-hopping commutator relations. This is justified by the trivial four-fold improvement of the sophisticated graph coloring algorithm compared to the brute-force coloring algorithm. However, for larger graphs produced by Number-to-Hopping commutators, the advantage of the sophisticated graph coloring algorithm is minimal. This is explainable by the tendency of Number-to-Hopping graphs to be more dense, but these explanations and results are only empirical. We leave a more detailed and theoretical justification to motivated readers in future work.

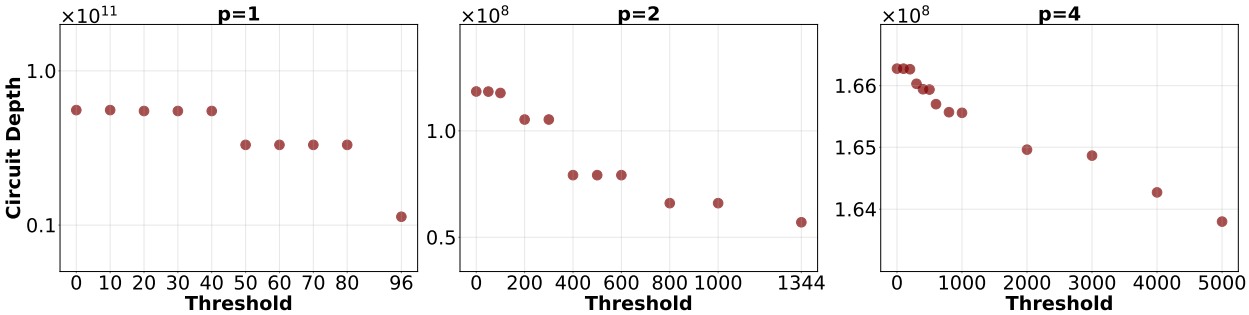

Figure 5: $p = 1, p = 2$, and $p = 4$ circuit depths with respect to threshold. We fix $\eta = 100$ to generate these plots. Note that the largest graph sizes for $p = 1, 2$ are 96 and 1344 respectively. [AZ: Add more ticks, make the dots bigger, make text larger, make the labels larger. Mention p=4 not much improvement, mention dips being correspondent to being in-between graph sizes. Beef up caption]

6 Discussion

In this paper, we described algorithms for computing product formulae error bounds of arbitrary translationally invariant k-local Hamiltonians that do not depend on the size of the lattice. In addition, we provided a python package that implements these algorithms. In this package, we also provided implementations for the Pauli operators, fermion strings, and the Pionless-EFT Hamiltonian. Finally, we apply our algorithms to the Pionless-EFT, and present experimental results to to show that product formula approximations beyond $p = 4$ are unnecessary in practice. In addition, we also present empirical comparisons between the brute-force and sophisticated graph coloring methods for the Pionless-EFT which suggest that the sophisticated graph coloring yields minimal returns as the size of the graph increases.

Several research topics related to our work can be further investigated. The nested-commutator scaling of the product formulae error bounds in order p are inherent to the problem. However, an additional theory on the bounds of the commutator expressions could be promising for more optimized algorithms. These theories would help us better analyze much higher-order product formulae $p = 6$ and generalize to other well-known nuclear Hamiltonians that are specified by translationally invariant operators. Implementation-specific improvements such as more clever data structures or smarter parallelization has the potential to drastically improve algorithm performance. In our algorithm, we rely heavily on multiple CPU parallelization and RAM to store exponentially expanding commutator computations. The exploration of parallelization across commutator-norm expressions entirely, contrary to our methodology, saves tremendous amounts of RAM usage. The performance of this parallelization remains unknown. Another direction for future work is to

analyze the application of our algorithm. Although $p = 2$ approximations remain the gold standard for smaller translationally invariant Hamiltonians specified by few terms, the effect remains unknown for larger translationally invariant Hamiltonians with much more base terms. The analysis we have shown for the crossover $p = 2$ and $p = 4$ computation suggests that similar crossover behavior can be shown for $p = 1$ and $p = 2$ computations. It is unknown how many base terms there must be to prove supremacy of $p = 1$ approximations, and consequently, whether $p = 1$ supremacy is practical for real world application.

References

[1] Guang Hao Low and Isaac L. Chuang. "Optimal Hamiltonian simulation by quantum signal processing". Physical Review Letters**118** (2017).

[2] Andrew M. Childs, Yuan Su, Minh C. Tran, Nathan Wiebe, and Shuchen Zhu. "Theory of Trotter error with commutator scaling". Phys. Rev. X **11**, 011020 (2021).

[3] James D Watson, Jacob Bringewatt, Alexander F Shaw, Andrew M Childs, Alexey V Gorshkov, and Zohreh Davoudi. "Quantum algorithms for simulating nuclear effective field theories" (2023). arXiv preprint arXiv:2312.05344.

[4] Yuan Su, Hsin-Yuan Huang, and Earl T. Campbell. "Nearly tight Trotterization of interacting electrons". Quantum **5**, 495 (2021).

[5] Masuo Suzuki. "General theory of fractal path integrals with applications to many-body theories and statistical physics". Journal of Mathematical Physics **32**, 400–407 (1991).

[6] Aric Hagberg, Pieter J. Swart, and Daniel A. Schult. "Exploring network structure, dynamics, and function using networkx". In Proceedings of the 7th Python in Science Conference (SciPy2008). Los Alamos National Laboratory (LANL) (2007). url: https://www.osti.gov/biblio/960616.

[7] Alessandro Roggero, Andy C. Y. Li, Joseph Carlson, Rajan Gupta, and Gabriel N. Perdue. "Quantum computing for neutrino-nucleus scattering". Physical Review D**101** (2020).

A Appendix

A.1 efficient decomposition of Pionless-EFT

We decompose the Pionless-EFT Hamiltonian as follows:

$$
\begin{aligned}
H_1 &\coloneqq -h \sum_x \sum_\sigma [a^\dagger_{2x}(\sigma) a_{2x+1}(\sigma) + a^\dagger_{2x+1}(\sigma) a_{2x}(\sigma)] \\
H_2 &\coloneqq -h \sum_x \sum_\sigma [a^\dagger_{2x+1}(\sigma) a_{2x+2}(\sigma) + a^\dagger_{2x+2}(\sigma) a_{2x+1}(\sigma)] \\
H_3 &\coloneqq -h \sum_y \sum_\sigma [a^\dagger_{2y}(\sigma) a_{2y+1}(\sigma) + a^\dagger_{2y+1}(\sigma) a_{2y}(\sigma)] \\
H_4 &\coloneqq -h \sum_y \sum_\sigma [a^\dagger_{2y+1}(\sigma) a_{2y+2}(\sigma) + a^\dagger_{2y+2}(\sigma) a_{2y+1}(\sigma)] \\
H_5 &\coloneqq -h \sum_z \sum_\sigma [a^\dagger_{2z}(\sigma) a_{2z+1}(\sigma) + a^\dagger_{2z+1}(\sigma) a_{2z}(\sigma)] \\
H_6 &\coloneqq -h \sum_z \sum_\sigma [a^\dagger_{2z+1}(\sigma) a_{2z+2}(\sigma) + a^\dagger_{2z+2}(\sigma) a_{2z+1}(\sigma)]
\end{aligned}
$$

$$
\begin{aligned}
H_7 \coloneqq 6h \sum_x \sum_\sigma N_x(\sigma) &+ \frac{C_0}{2} \sum_x \sum_{\sigma \neq \sigma'} N_x(\sigma) N_x(\sigma') \\
&+ \frac{D_0}{6} \sum_x \sum_{\sigma \neq \sigma' \neq \sigma''} N_x(\sigma) N_x(\sigma') N_x(\sigma'')
\end{aligned}
$$

In total, we have

$$
d \times 2 \times s + \binom{s}{3} + \binom{s}{2} + s \tag{32}
$$

where d is the number of dimensions, and s is the number of spins. For the 3D Pionless-EFT, plugging in $d = 3$ and $s = 4$ yields 38 base terms.